

Today's Session

U:PASS Student Feedback Survey

U:PASS Feedback for Autumn 2025



<https://forms.office.com/r/UfASx79NXj>

Student Feedback Survey



Machine Learning
Algorithms and
Applications

<https://go.blueja.io/KxnqfkWtL0i0FXpAAOwj5A>

- Completely anonymous
- All feedback are welcomed

A screenshot of the UTS online Student Feedback Survey (SFS) login page. The page has a blue header with the 'EARLY FEEDBACK SURVEY' and 'STUDENT FEEDBACK SURVEY' logos. Below the header, there's a status bar showing 'Open online surveys: 42' and 'Current participation: 14.94%'. The main content area is titled 'Welcome to the UTS online Student Feedback Survey (SFS)' and contains instructions for students and staff. There are two login sections: one for 'STUDENT' and one for 'STAFF', each with fields for 'UTS ID' and 'UTS password' and a 'GO' button. The footer includes 'UTSOnline', 'One Stop Student Admin', and 'v2.4.1'.

Machine Learning Algorithms and Applications

Lecture 10

(Block 4)

Plan for Today



Data In Production



Machine Learning Lifecycle



Model Artefact



Model Serving with Flask



Lab



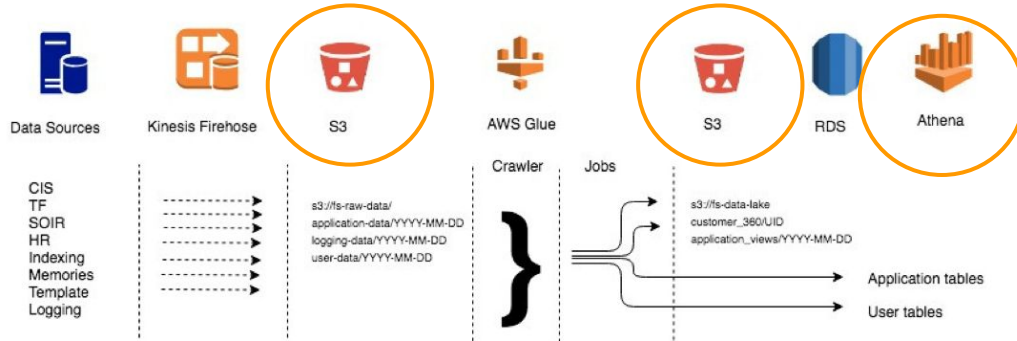
Data in Production

Data Sources

	Datalake	Data warehouse	Datamart	API
Description	Raw data Unstructured Unprocessed Distributed	Curated data Schema Cleaned	Curated data Subset	External data On-request
Format	CSV Parquet Avro ...	Redshift Snowflake Postgres MySQL MongoDB ...	Postgres MySQL MongoDB ...	JSON Text CSV ...

Datalake

Repository that stores data in raw format (ie files). The goal is to store all information without worrying about quality or business value.



Example of data architecture with AWS

Python Packages for Cloud Services

Boto3 documentation

Boto is the Amazon Web Services (AWS) SDK for Python. It enables Python developers to create, configure, and manage AWS services, such as EC2 and S3. Boto provides an easy to use, object-oriented API, as well as low-level access to AWS services.

Quickstart

Python Packages

Idiomatic Client Libraries for Google Cloud Platform Service APIs

google-cloud-bigquery
Python client library for the BigQuery API.

google-cloud-bigtable
Python client library for the Cloud Bigtable API.

google-cloud-datastore
Python client library for the Cloud Datastore API.

Use the Azure libraries (SDK) for Python

05/26/2020 • 4 minutes to read • 📖

The open-source Azure libraries for Python simplify provisioning, managing, and using Azure resources from Python application code.

The details you really want to know

- The Azure libraries are how you communicate with Azure services from Python code that you run either locally or in the cloud. (Whether you can run Python code within the scope of a particular service depends on whether that service itself currently supports Python.)
- The libraries support Python 2.7 and Python 3.5.3 or later, and it tested also with PyPy 5.4+.
- The Azure SDK for Python is composed solely of over 180 individual Python libraries that relate to specific Azure services. There are no other tools in the "SDK".

AWS S3



S3

Read data from flat files directly from
a S3 bucket

Requires following environment variable

- `aws_access_key_id`
- `aws_secret_access_key`
- `aws_session_token`

boto3

```
import boto3
```

```
s3 = boto3.resource('s3')  
obj = s3.Object('my_bucket_name', 'my_prefix_name/my_filename')  
body = obj.get()['Body'].read()
```

fastavro

```
import fastavro  
  
with open(filepath) as fp:  
    reader = fastavro.reader(fp)  
    records = [r for r in reader]
```

pandas

```
import pandas as pd
```

```
df = pd.read_csv("s3:/my_bucket_name/my_prefix_name/my_filename")  
df = pd.read_parquet("s3:/my_bucket_name/my_prefix_name/my_filename")
```

AWS Athena



Read data with SQL-like queries
from a S3 bucket

Athena

Requires following environment variable

- aws_access_key_id
- aws_secret_access_key
- aws_session_token

boto3

```
import boto3
```

```
client = boto3.client('athena')
```

```
queryStart = client.start_query_execution(  
    QueryString = 'SELECT * FROM myTable',  
    QueryExecutionContext = {  
        'Database': 'myDatabase'  
    }  
)
```

```
while state in ['RUNNING', 'QUEUED']:  
    response = client.get_query_execution(QueryExecutionId = queryStart.QueryExecutionId)  
  
    if 'QueryExecution' in response and \  
        'Status' in response['QueryExecution'] and \  
        'State' in response['QueryExecution']['Status']:  
        state = response['QueryExecution']['Status']['State']  
        if state == 'SUCCEEDED':  
            results = client.get_query_results(queryStart.QueryExecutionId)  
            time.sleep(1)
```

Datawarehouse/database/datamart

Datawarehouse: Central repository that stores curated data in a defined schema. It is mainly used for data analysis and reporting.

Datamart: Subset of datawarehouse accessible for a specific business team.

Database: Collection of structured data. It is usually related to a running application.



Database



```
import psycopg2
```

```
con = psycopg2.connect(  
    user = "postgres",  
    password = "1234",  
    host = "localhost",  
    port = "5432",  
    database = "demo"  
)
```

```
cursor = con.cursor()  
cursor.execute("SELECT * FROM table")  
rows = cursor.fetchall()  
con.close()
```



```
import mysql.connector as mysql
```

```
con = mysql.connect(  
    user = "mysql",  
    passwd = "1234",  
    host = "localhost",  
    database = "demo"  
)
```

```
cursor = con.cursor()  
cursor.execute("SELECT * FROM table")  
rows = cursor.fetchall()  
con.close()
```



```
import pymongo
```

```
myclient = pymongo.MongoClient("mongodb://localhost:27017/")
```

```
mydb = myclient["mydatabase"]  
mycol = mydb["customers"]  
myquery = { "address": "15 Broadway, Ultimo" }  
mydoc = mycol.find(myquery)
```

```
graph LR
    subgraph Client
        C[Client]
    end
    subgraph Server
        subgraph API
            GE[GET Endpoint]
            PE[POST Endpoint]
        end
        DB[(Database)]
    end
    C -- "GET Request" --> GE
    GE -- "Response Data" --> C
    GE -- "Read" --> DB
    C -- "POST Request" --> PE
    PE -- "Response Data" --> C
    PE -- "Update" --> DB
```

```
response = requests.get('https://api.github.com/search/repositories?q=python')
```

200

```
{ "total_count":1600315,"complete_results":false,"items":[{"id":"63476337","node_id":"MDEwOjUlcG9zaXRvcnNzMzE","owner":{"login":"TheAlgorithms","id":"20487725","node_id":"MDEyOk9yZTFlbnVudG93NmI1","avatar_url":"https://api.github.com/users/TheAlgorithms","html_url":"https://api.github.com/users/TheAlgorithms","followers_url":"https://api.github.com/users/TheAlgorithms/following/{other_user}","gists_url":"https://api.github.com/users/TheAlgorithms/gists/{owner}/{repo}","subscriptions_url":"https://api.github.com/users/TheAlgorithms/subscriptions/{owner}/{repo}","events_url":"https://api.github.com/users/TheAlgorithms/repos/{owner}/{repo}/events"}]}
```

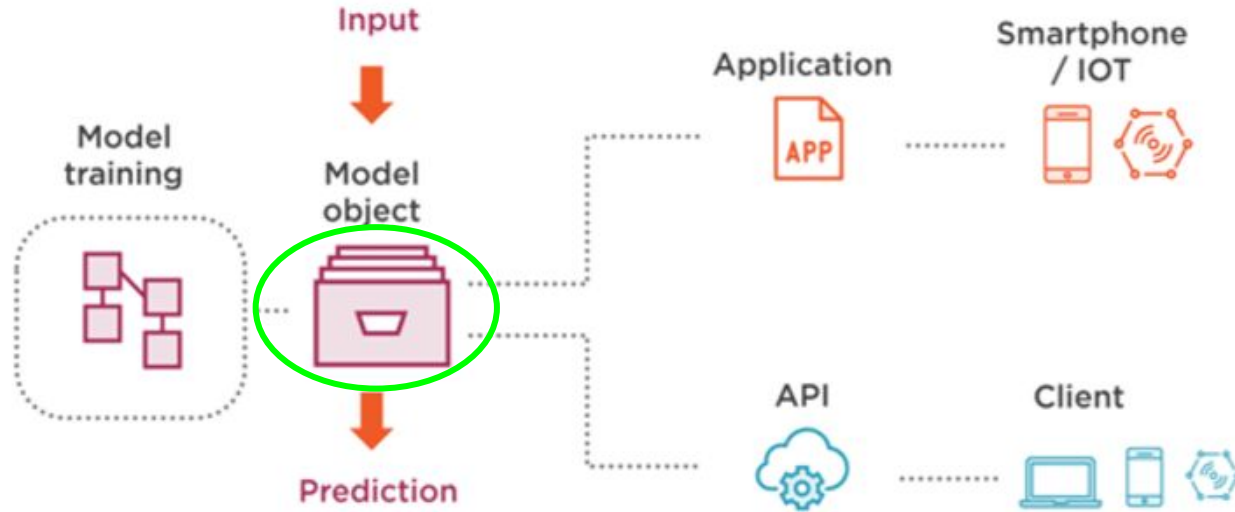
```
json.loads(response.text)
```

```
{
  'incomplete_results': False,
  'items': [
    {
      'archive_url': 'https://api.github.com/repos/TheAlgorithms/Python/{archive_format}/{ref}'.format(archive_format='zip', ref='main'),
      'archived': False,
      'assignees_url': 'https://api.github.com/repos/TheAlgorithms/Python/assignees/{user}'.format(user='TheAlgorithms'),
      'blobs_url': 'https://api.github.com/repos/TheAlgorithms/Python/git/blobs/{sha}'.format(sha='main'),
      'branches_url': 'https://api.github.com/repos/TheAlgorithms/Python/branches/{branch}'.format(branch='main'),
      'clone_url': 'https://github.com/TheAlgorithms/Python.git',
      'collaborators_url': 'https://api.github.com/repos/TheAlgorithms/Python/collaborators/{collaborator}'.format(collaborator='TheAlgorithms'),
      'comments_url': 'https://api.github.com/repos/TheAlgorithms/Python/comments/{number}'.format(number='1'),
      'commits_url': 'https://api.github.com/repos/TheAlgorithms/Python/commits/{sha}'.format(sha='main'),
      'compare_url': 'https://api.github.com/repos/TheAlgorithms/Python/compare/{base}...{head}'.format(base='main', head='main')
    }
  ]
}
```



Saving Model

Beyond Model Training



Pickle: Save and Load Trained Models

- **Save model to a file**

```
from sklearn import svm

clf = svm.SVC()
clf.fit(X, y)

import pickle

outfile = open('model_svc.pkl', 'wb')
pickle.dump(clf, outfile)
outfile.close()
```

- **Load model from file**

```
infile = open('model_svc.pkl', 'rb')
clf2 = pickle.load(infile)
infile.close()

clf2.predict(X)
```

Joblib: Save and Load Trained Models

- Save model to a file

```
from sklearn import svm

clf = svm.SVC()
clf.fit(X, y)

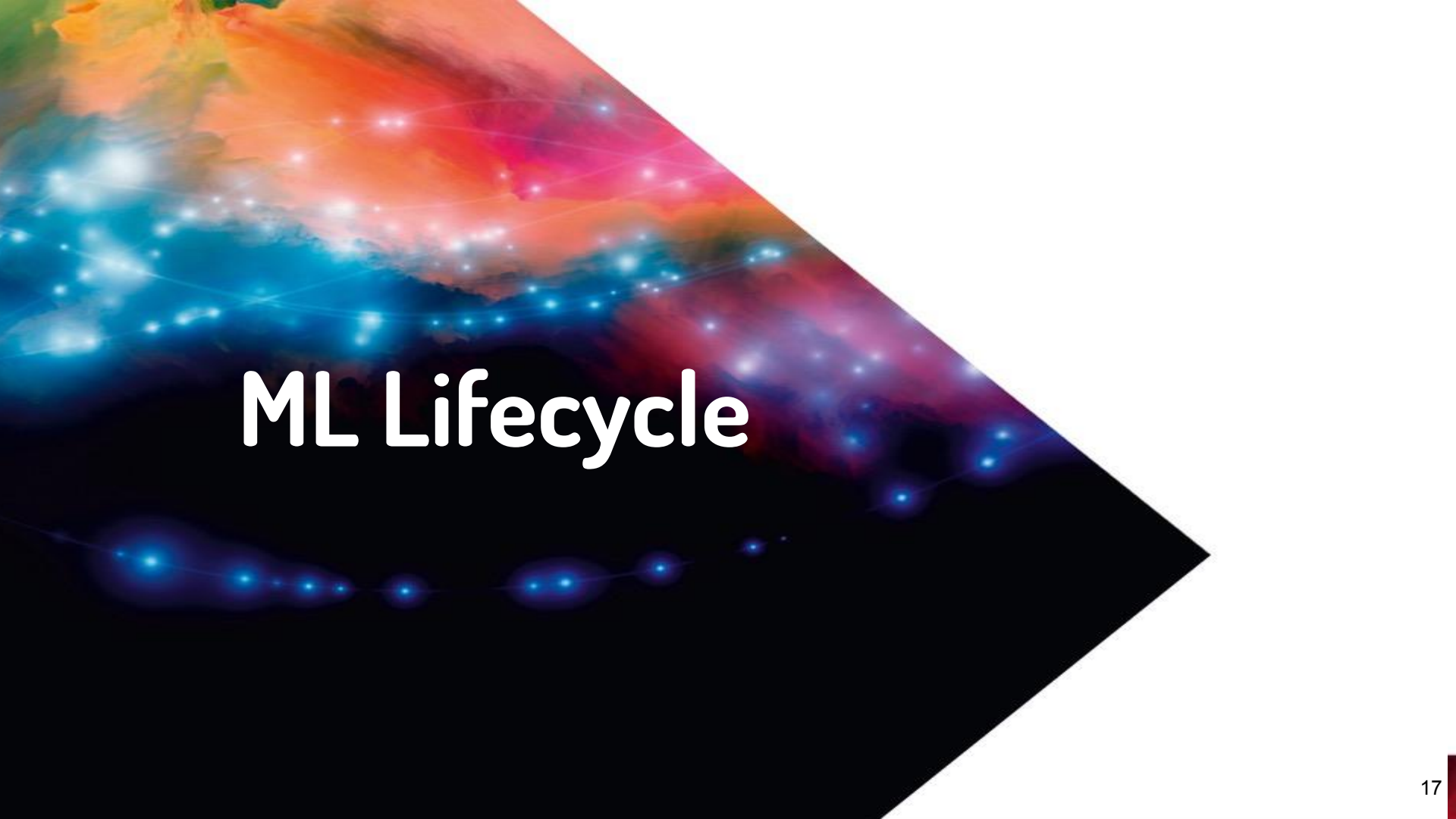
from joblib import dump

dump(clf, 'model_svc.joblib')
```

- Load model from file

```
from joblib import load

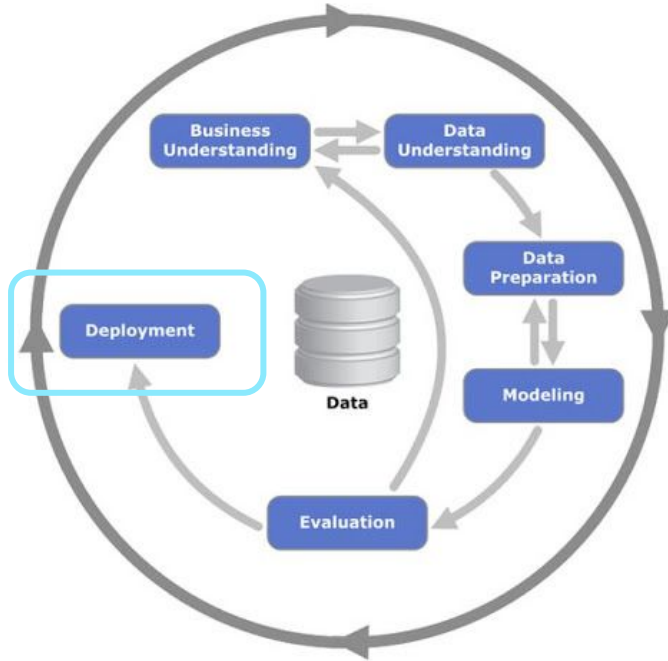
clf2 = load('model_svc.joblib')
clf2.predict(X)
```



ML Lifecycle

Data Science Project

CRISP-DM



Business Understanding

Understanding of business objectives and requirements and converting them into data mining problems

Data Understanding

Data collection and analysis for identifying data quality issues, discovering initial insights and defining hypotheses

Data Preparation

Performing required preparation tasks on the data (data cleaning, feature engineering and set splitting) to construct final dataset for modeling stage

Modeling

Training of models and hyperparameters tuning

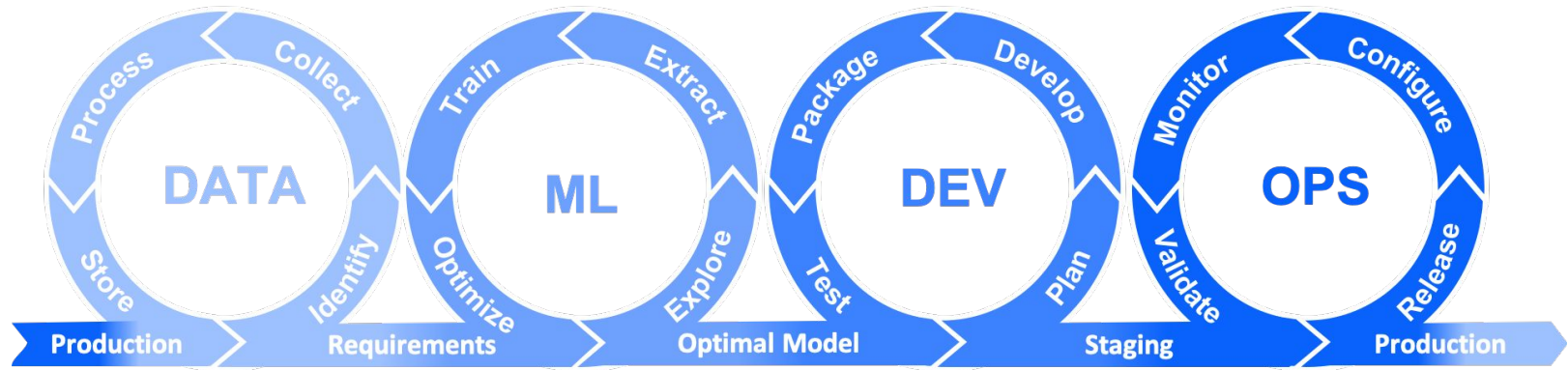
Evaluation

Assessment of the models performance and generalisation capabilities on unseen data

Deployment

Deployment of models and required pipelines into production environment for serving business use case

ML Lifecycle



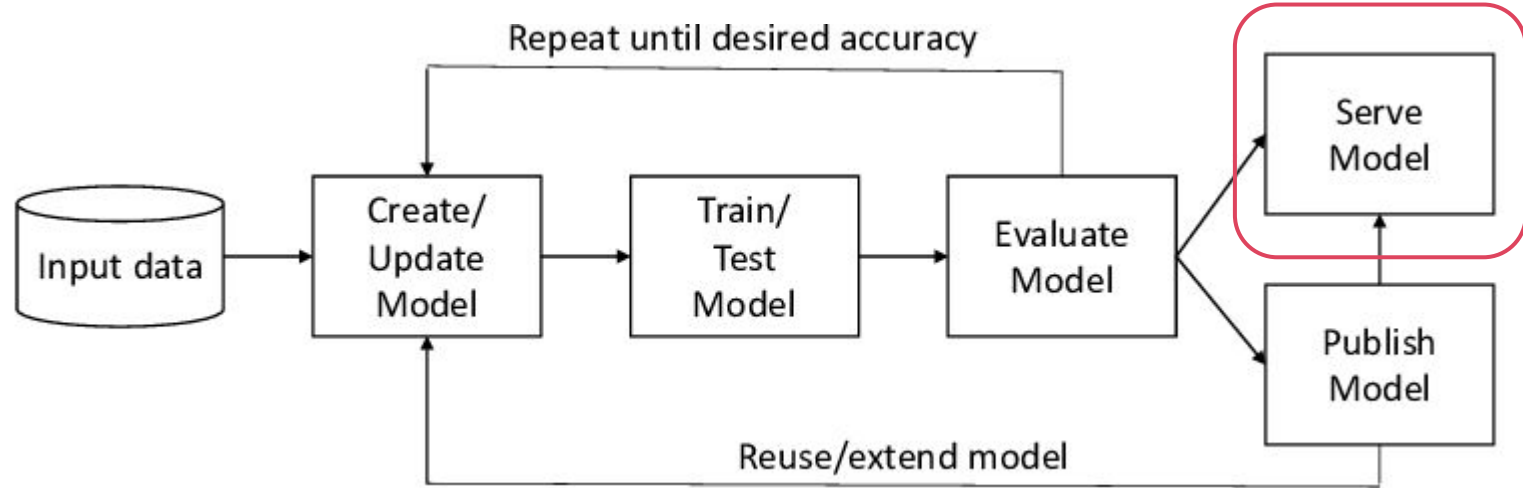
Management of Data
(ETL, storage, catalogue...)

Data Science Experiment (feature engineering, model training, hyperparameter tuning, evaluation...)

Model Serving Development
(saving model, pipeline, API...)

Model Operation
(infrastructure, versioning, tracking, CI/CD...)

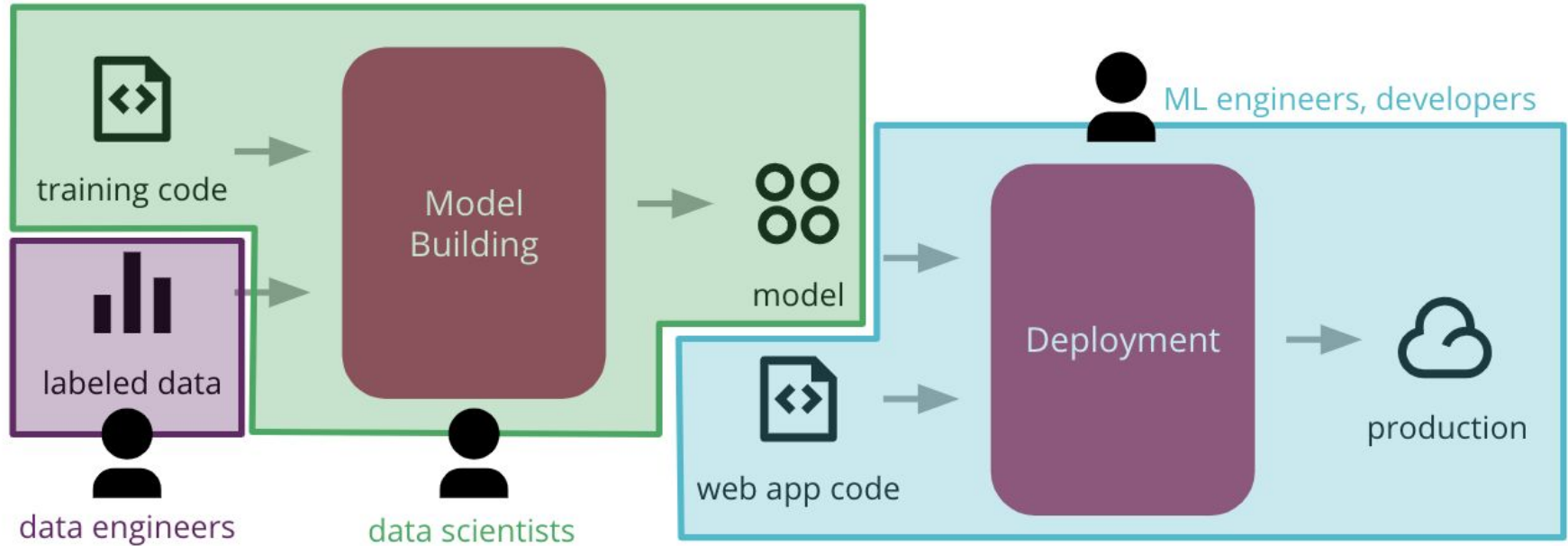
ML Model Lifecycle



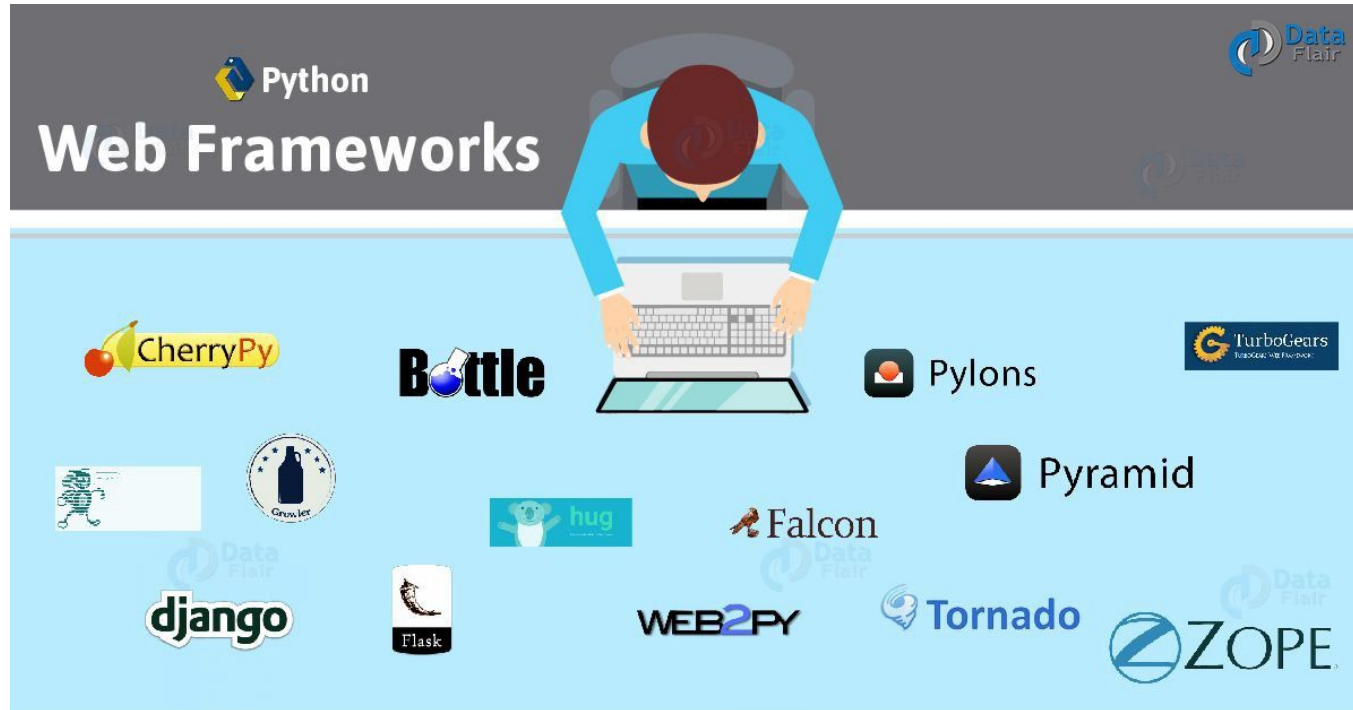


Web Application

Training vs Inference

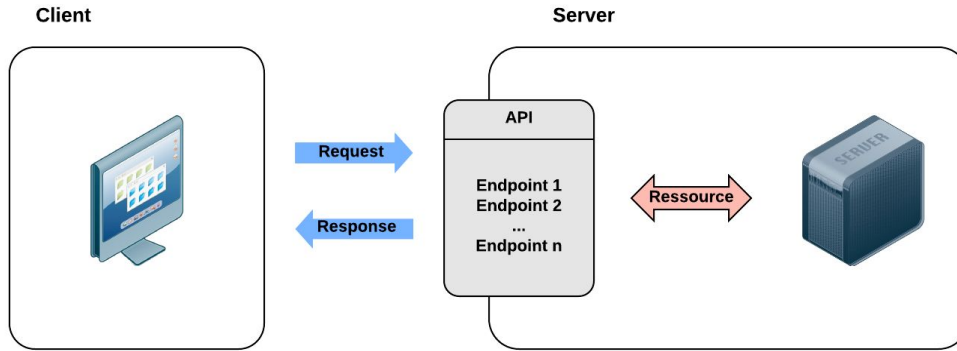


Web App Frameworks



HTTP Requests

API (Application Programming Interface)

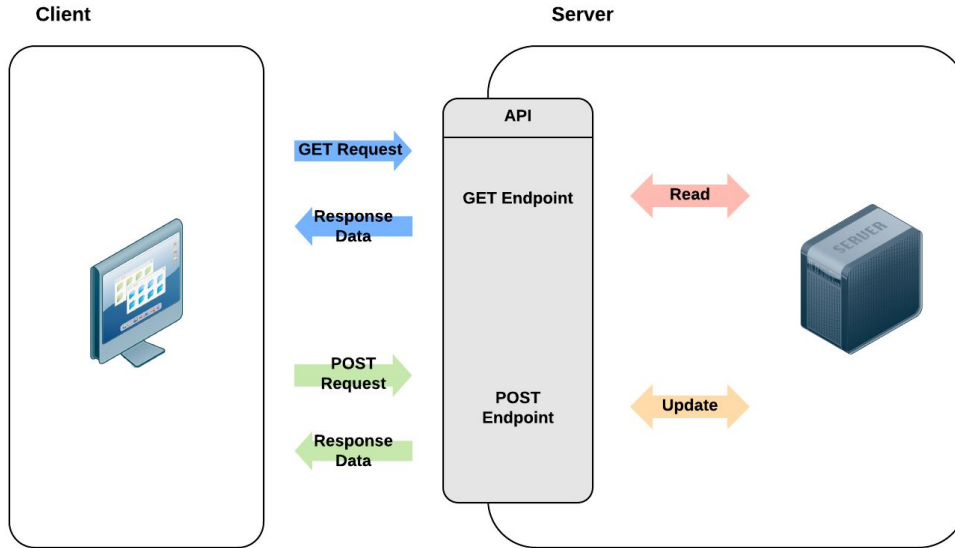


Client (web browser) makes a HTTP request to a server

The request is made through specific API endpoint
(<https://www.myserver.com/customer/churn/prediction>)

The server processes the request and send back the results as JSON

HTTP POST and GET Requests



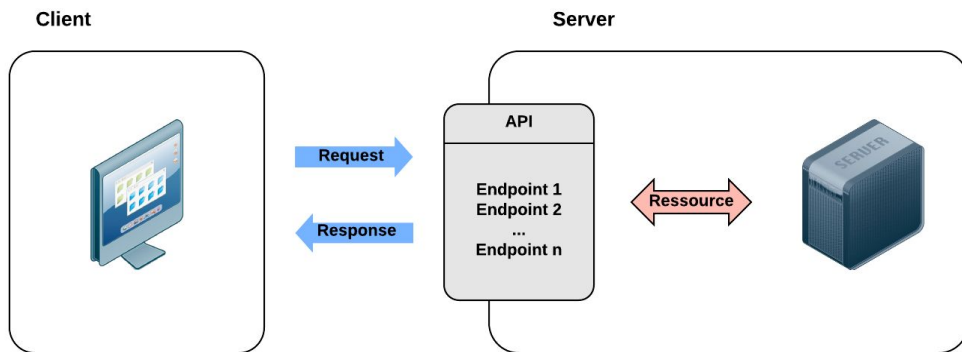
Multiple type of HTTP requests: GET, POST, DELETE, UPDATE...

GET: retrieve some resources from the server

POST: provide resources to be stored on the server

Model Serving with HTTP Requests

API (Application Programming Interface)

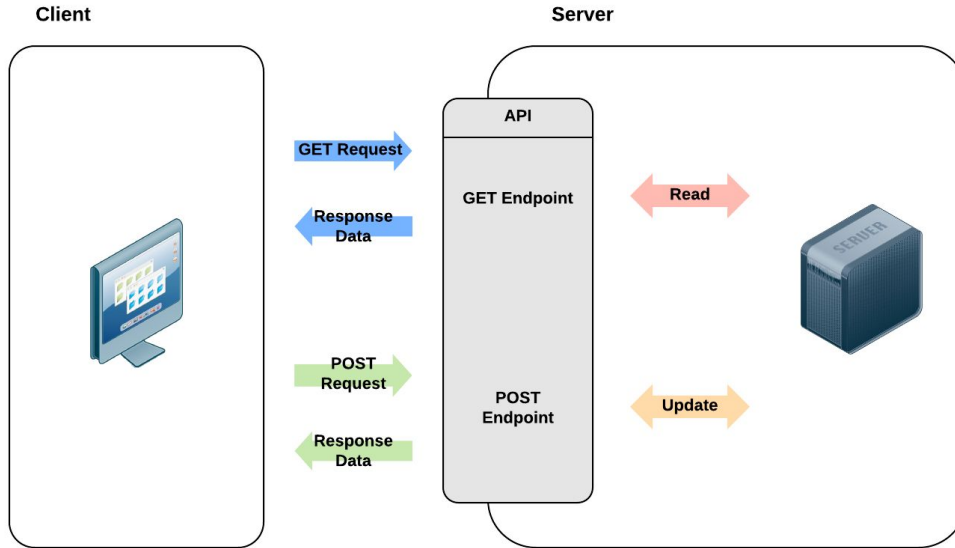


Client (web browser) makes a HTTP request to a server

The request is made through specific API endpoint
(<https://www.myserver.com/customer/churn/prediction>)

The server processes the request and send back the results as JSON

HTTP POST and GET Requests



Multiple type of HTTP requests: GET, POST, DELETE, UPDATE...

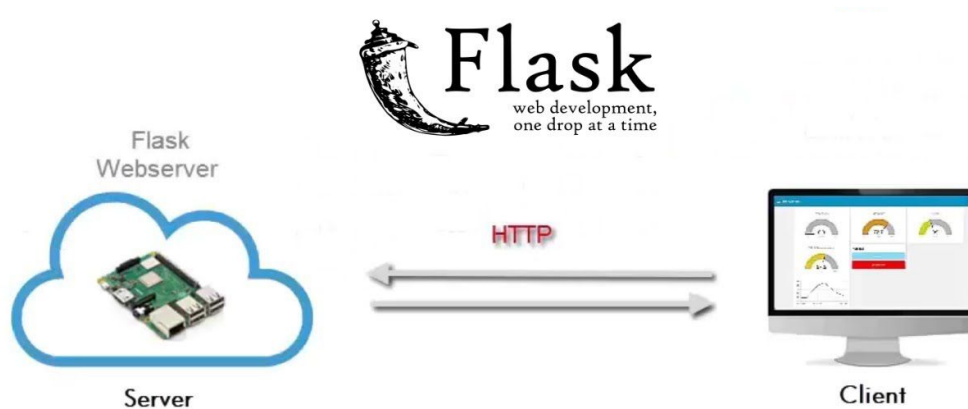
GET: retrieve some resources from the server

POST: provide resources to be stored on the server



Flask App

Flask Framework



Micro web framework in Python

Used for creating Web application or Web API

Very simple to use, reliable, scalable

Can be extended with additional packages

Setup Flask

Install Flask

```
pip install Flask
```

Create a directory called app

```
mkdir app
```

Create a Python script called
app/main.py

```
nano app/main.py
```

Create Flask App

Inside app/main.py

Import package

```
from flask import Flask
```

Create Web Application

```
app = Flask(__name__)
```

Add Flask GET endpoint

Add endpoint with constructor 'route'

GET endpoint for root
(Python decorator)

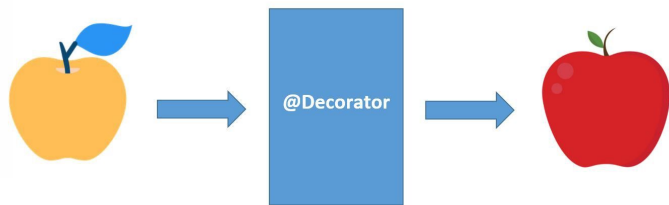
```
@app.route("/")  
def welcome():  
    return "It works!"
```

Function that is linked
to the endpoint

Business logic and
JSON output

Python Decorator

Decorators can be extremely useful as they allow the extension of an existing function, without any modification to the original function source code.



```
@app.route("/")
def welcome():
    return "It works!"
```

`@app.route("/")` is a Python decorator that Flask provides to assign URLs in our app to functions easily.

The decorator is telling our `@app` that whenever a user visits our app domain at the given `.route()`, execute the `welcome()` function.

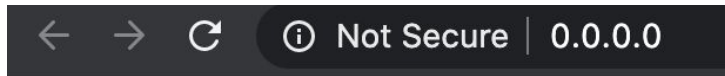
Launch Flask App

Launch Web Application

```
$ export FLASK_APP=main
```

```
$ flask run
```

Open a browser on 0.0.0.0



It works!

Add Flask POST endpoint

POST endpoint for root
(Python decorator)

Add endpoint

```
@app.route('/items', methods=['POST'])
def add():
    data = request.get_json()
    df = pd.DataFrame(data, index=[0])
    return df
```

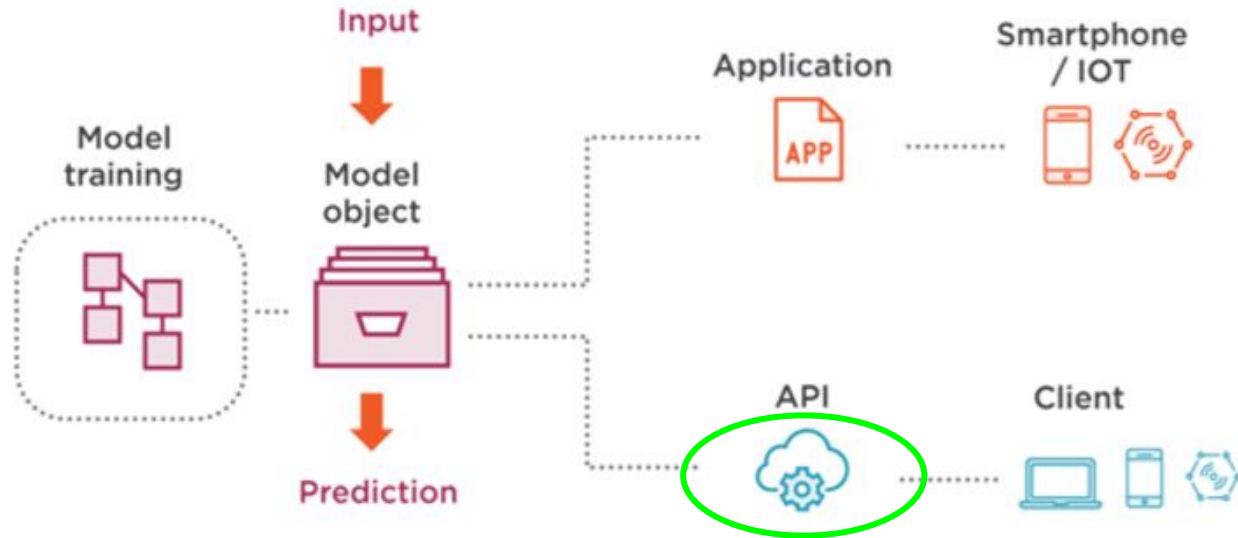
Function that is linked
to the endpoint with
input parameter

Business logic and
JSON output



ML API with Flask

Model Serving



ML endpoint with Flask

Load trained model

```
import joblib

model = joblib.load("model.pkl")
```

Add endpoint for
prediction and
conversion to string

```
import numpy as np

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    prediction = model.predict(data)
    str_pred = np.array2string(prediction)
    return jsonify(str_pred)
```

Flask on Google Colab



Google doesn't provide public IP addresses for their Colab servers => can't access the API externally

Solution: Create a tunnel that expose Colab via a public IP address

- Create a free [ngrok](https://ngrok.io) account
- Build Flask App on Colab
- Create ngrok tunnel for Flask app
- Launch Flask App
- Send requests to the ngrok IP address

Ngrok Setup on Google Colab

1. Authenticate with ngrok

```
from pyngrok import ngrok, conf  
  
import getpass  
conf.get_default().auth_token = getpass.getpass()
```

2. Create ngrok Tunnel

```
public_url = ngrok.connect(5000).public_url
```

3. Configure Flask IP

```
app.config["BASE_URL"] = public_url
```

Launch Flask on Google Colab

1. Create Flask app

```
from flask import Flask

app = Flask("my_app")

@app.route("/")
def welcome():
    return "Welcome to My App"
```

2a. Launch Flask app

```
app.run()

* Serving Flask app "__main__" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

2b. Launch Flask app as separate thread

```
import threading

flask_thread = threading.Thread(target=app.run, kwargs={"use_reloader": False})
flask_thread.start()

* Serving Flask app "rf" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

Requests Package

Import package:

```
import requests
```

Make GET call:

```
response = requests.get("https://jsonplaceholder.com/get")
```

Make POST call:

```
response = requests.post("https://jsonplaceholder.com/post", data=data)
```

Extract status code:

```
response.status_code
```

Extract API content as text:

```
response.text
```

Extract API content as dictionary:

```
response.json()
```

Sending API Requests

Send GET Request

```
import requests
```

```
r = requests.get(f"http://{ip_address}")  
r.text
```

```
172.28.0.2 - - [26/Jul/2020 10:33:11] "GET / HTTP/1.1" 200 -  
'Welcome to my API!'
```

Send POST Request with parameter

```
import requests
```

```
import json
```

```
data = {'col1':[0,1,2], 'col2':[3,4,5]}
```

```
headers = {'content-type': 'application/json', 'Accept-Charset': 'UTF-8'}
```

```
r = requests.post(f"http://{ip_address}/data",  
                  data=json.dumps(data), headers=headers)
```

```
r.text
```

```
172.28.0.2 - - [26/Jul/2020 10:56:28] "POST /data HTTP/1.1" 200 -  
'{"col1":[0,1,2],"col2":[3,4,5]}\n'
```



Time for Lab





What's Next?

MDSI Electives (Spring)

First year

36101 Leading Data Science Initiatives

Project management
Agile
Uncertainty with data

94692 Data Science Practise

Database
SQL
Data Analysis with Python
Building Data App

Second year

36118 Applied Natural Language Processing

Text Processing
NLP
NLU
NLG

94693 Big Data Engineering

Datalake
Datawarehouse
Batch Processing
Big data
Streaming
ML at scale

36120 Advanced ML Application

Advanced algorithms
MLOps
Model interpretation
Docker
Custom package
Professional project



MDSI Electives (Autumn)

First year

**36121 Artificial Intelligence
Principles and Applications**

Project management
Agile
Uncertainty with data

36122 Python Programming

Software Engineering
Python

Second year

**36118 Applied Natural
Language Processing**

Text Processing
NLP
NLU
NLG

94691 Deep Learning

Neural Networks
Image Processing
NLP
Advanced Architecture





Final Words

MLAA

Learning Experience



Learn by doing

Hands-on approach
Gradual learning curves



Real problems

Real world dataset and business
problems
Best practices from industry



Human-centered approach

Deal with uncertainty
Ethics and integrity
Responsible AI



Innovation

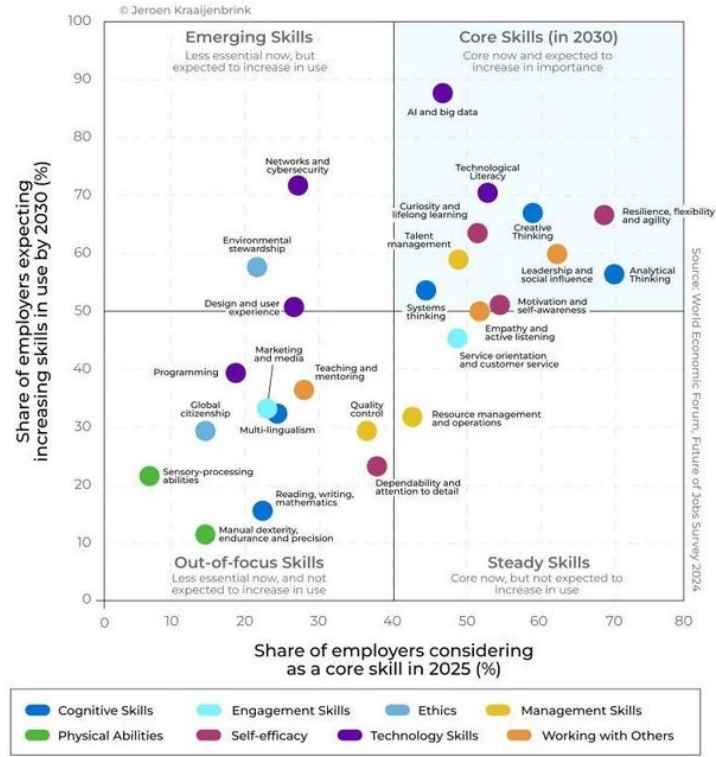
Problem solving
Challenges
Creative thinking



The Core Skills for 2030

Report (7 January 2025):

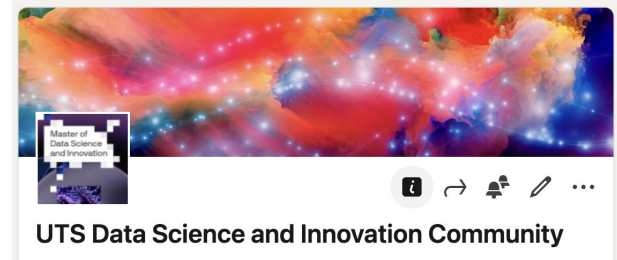
<https://www.weforum.org/publications/the-future-of-jobs-report-2025/>



Stay in touch



LinkedIn profile



LinkedIn private group

